In [1]:
```
import pandas as pd
df=pd.read_csv("test_Y3wMUE5_7gLdaTN.csv")
df.head(10)
```

Out[1]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Prope
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.0	
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.0	
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.0	
3	LP001035	Male	Yes	2	Graduate	No	2340	2546	100.0	360.0	NaN	
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.0	
5	LP001054	Male	Yes	0	Not Graduate	Yes	2165	3422	152.0	360.0	1.0	
6	LP001055	Female	No	1	Not Graduate	No	2226	0	59.0	360.0	1.0	Se
7	LP001056	Male	Yes	2	Not Graduate	No	3881	0	147.0	360.0	0.0	
8	LP001059	Male	Yes	2	Graduate	NaN	13633	0	280.0	240.0	1.0	
9	LP001067	Male	No	0	Not Graduate	No	2400	2400	123.0	360.0	1.0	Se

jupyter Loan prediction Last Checkpoint: 4 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab

[2]:
```
#used to identify missing values
print(df.isnull())

# count missing values for all field
print(df.isnull().sum())
```

```
     Loan_ID  Gender  Married  Dependents  Education  Self_Employed  \
0      False   False    False       False      False          False
1      False   False    False       False      False          False
2      False   False    False       False      False          False
3      False   False    False       False      False          False
4      False   False    False       False      False          False
..       ...     ...      ...         ...        ...            ...
362    False   False    False       False      False          False
363    False   False    False       False      False          False
364    False   False    False       False      False          False
365    False   False    False       False      False          False
366    False   False    False       False      False          False

     ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0              False              False       False             False
1              False              False       False             False
2              False              False       False             False
3              False              False       False             False
4              False              False       False             False
..               ...                ...         ...               ...
362            False              False       False             False
363            False              False       False             False
364            False              False       False             False
365            False              False       False             False
366            False              False       False             False

     Credit_History  Property_Area
0             False          False
1             False          False
2             False          False
```

```
[367 rows x 12 columns]
Loan_ID               0
Gender               11
Married               0
Dependents           10
Education             0
Self_Employed        23
ApplicantIncome       0
CoapplicantIncome     0
LoanAmount            5
Loan_Amount_Term      6
Credit_History       29
Property_Area         0
dtype: int64
```

[3]:

```
# Fill Self_Employed with mode
mode_self_employed = df['Self_Employed'].mode()[0]
df['Self_Employed'] = df['Self_Employed'].fillna(mode_self_employed)

# Fill Credit_History with mode
mode_credit = df['Credit_History'].mode()[0]
df['Credit_History'] = df['Credit_History'].fillna(mode_credit)

mode_gender = df['Gender'].mode()[0]
df['Gender'] = df['Gender'].fillna(mode_gender)

mode_dependents = df['Dependents'].mode()[0]
df['Dependents'] = df['Dependents'].fillna(mode_dependents)

mode_amount = df['LoanAmount'].mode()[0]
df['LoanAmount'] = df['LoanAmount'].fillna(mode_amount)

mode_term= df['Loan_Amount_Term'].mode()[0]
df['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(mode_term)
# Check missing values
```

```
Loan_ID              0
Gender               0
Married              0
Dependents           0
Education            0
Self_Employed        0
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount           0
Loan_Amount_Term     0
Credit_History       0
Property_Area        0
dtype: int64
```

[4]:
```
print(df.columns)
```

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area'],
      dtype='object')
```

[13]:
```
df.rename(columns={
    'ApplicantIncome':'Applicant_Income'
},inplace=True)
print(df.columns)
```

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'Applicant_Income', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area'],
      dtype='object')
```

[14]:
```
#3. Changing Data Types
#check Data Types.
print(df.dtypes)
```

```
Loan_ID              object
```

```
Loan_ID              object
Gender               object
Married              object
Dependents           object
Education            object
Self_Employed        object
Applicant_Income      int64
CoapplicantIncome     int64
LoanAmount          float64
Loan_Amount_Term    float64
Credit_History      float64
Property_Area        object
dtype: object
```

[15]:
```
#4. Detecting Duplicate Records
#Find Duplicates
duplicates=df.duplicated()
print(duplicates)
```

```
0      False
1      False
2      False
3      False
4      False
       ...
362    False
363    False
364    False
365    False
366    False
Length: 367, dtype: bool
```

[16]:
```
#Count duplicates
print(df.duplicated().sum())
```

```
0
```

[17]:
```
#5. Dropping duplicate records
#Before
print(df.shape)
#Remove duplicates
df.drop_duplicates(inplace=True)
#After
print(df.shape)
```

```
(367, 12)
(367, 12)
```

[12]:
```
# groupby
df.groupby('Gender')['LoanAmount'].mean()
#agg
df.groupby('Education').agg({
    'LoanAmount':['mean','min','max']
})

# transform()
df['Avg_Loan_By_Education'] = df.groupby('Education')['LoanAmount'].transform('mean')
print(df)
```

```
      Loan_ID Gender Married Dependents     Education Self_Employed  \
0    LP001015   Male     Yes          0      Graduate            No
1    LP001022   Male     Yes          1      Graduate            No
2    LP001031   Male     Yes          2      Graduate            No
3    LP001035   Male     Yes          2      Graduate            No
4    LP001051   Male      No          0  Not Graduate            No
..        ...    ...     ...        ...           ...           ...
362  LP002971   Male     Yes         3+  Not Graduate           Yes
363  LP002975   Male     Yes          0      Graduate            No
364  LP002980   Male      No          0      Graduate            No
365  LP002986   Male     Yes          0      Graduate            No
366  LP002989   Male      No          0      Graduate           Yes

     ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0               5720                  0       110.0             360.0
```

```
362             4009               1777       113.0             360.0
363             4158                709       115.0             360.0
364             3250               1993       126.0             360.0
365             5000               2393       158.0             360.0
366             9200                  0        98.0             180.0

     Credit_History Property_Area  Avg_Loan_By_Education
0               1.0         Urban              141.480565
1               1.0         Urban              141.480565
2               1.0         Urban              141.480565
3               1.0         Urban              141.480565
4               1.0         Urban              118.940476
..              ...           ...                     ...
362             1.0         Urban              118.940476
363             1.0         Urban              141.480565
364             1.0     Semiurban              141.480565
365             1.0         Rural              141.480565
366             1.0         Rural              141.480565

[367 rows x 13 columns]
```

[3]:

```
# Average Loan Amount by Gender
pivot1 = pd.pivot_table(
    df,
    values='LoanAmount',
    index='Gender',
    aggfunc='mean'
)

print(pivot1)

# Average Loan Amount by Education
pd.pivot_table(
    df,
    values='LoanAmount',
    index='Education',
```

```
    df,
    values='LoanAmount',
    index='Education',
    aggfunc='mean'
)


# Average Loan Amount by Property Area
pd.pivot_table(
    df,
    values='LoanAmount',
    index='Property_Area',
    aggfunc='mean'
)
```

```
            LoanAmount
Gender
Female      126.797101
Male        139.031915
```

[3]:

Property_Area	LoanAmount
Rural	138.181818
Semiurban	134.043860
Urban	136.224638

[18]:

```
print(df.columns)
```

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'Applicant_Income', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area'],
      dtype='object')
```

data Preprocessing

[19]:
```
import pandas as pd
import seaborn as sns
#Load Loan prediction dataset
df=pd.read_csv("test_Y3wMUE5_7gLdaTN.csv")
print(df.head())
print(df.shape)
```

```
    Loan_ID Gender Married Dependents     Education Self_Employed  \
0  LP001015   Male     Yes          0      Graduate            No
1  LP001022   Male     Yes          1      Graduate            No
2  LP001031   Male     Yes          2      Graduate            No
3  LP001035   Male     Yes          2      Graduate            No
4  LP001051   Male      No          0  Not Graduate            No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             5720                  0       110.0             360.0
1             3076               1500       126.0             360.0
2             5000               1800       208.0             360.0
3             2340               2546       100.0             360.0
4             3276                  0        78.0             360.0

   Credit_History Property_Area
0             1.0         Urban
1             1.0         Urban
2             1.0         Urban
3             NaN         Urban
4             1.0         Urban
(367, 12)
```

[20]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
```

```
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Loan_ID            367 non-null    object
 1   Gender             356 non-null    object
 2   Married            367 non-null    object
 3   Dependents         357 non-null    object
 4   Education          367 non-null    object
 5   Self_Employed      344 non-null    object
 6   ApplicantIncome    367 non-null    int64
 7   CoapplicantIncome  367 non-null    int64
 8   LoanAmount         362 non-null    float64
 9   Loan_Amount_Term   361 non-null    float64
 10  Credit_History     338 non-null    float64
 11  Property_Area      367 non-null    object
dtypes: float64(3), int64(2), object(7)
memory usage: 34.5+ KB
```

[21]:
```
df.loc[0, 'Gender'] = 'male'
df.loc[1, 'Gender'] = 'MALE'
df.loc[2, 'Gender'] = 'Male'
print(df['Gender'].unique())
```

```
['male' 'MALE' 'Male' 'Female' nan]
```

[22]:
```
df['Gender'] = df['Gender'].str.title()   # all the words 1st letter will be in uppercase
print(df['Gender'].unique())
```

```
['Male' 'Female' nan]
```

[23]:
```
print(df['Gender'].isnull().sum())
```

```
11
```

[24]:
```
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
print(df['Gender'].unique())
```

```
['Male' 'Female']
```

jupyter Loan prediction Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab

[25]:
```
df = df.drop_duplicates()
```

[26]:
```
# check duplicates
print(df.duplicated().sum())

# remove duplicates
df = df.drop_duplicates()

# check shape
print(df.shape)
```

```
0
(367, 12)
```

[27]:
```
df['Gender'] = df['Gender'].replace({
    'Mle': 'Male',
    'male': 'Male',
    'MALE': 'Male'
})
print(df['Gender'].value_counts())
```

```
Gender
Male      297
Female     70
Name: count, dtype: int64
```

[28]:
```
print(df['Gender'].unique())
```

```
['Male' 'Female']
```

[29]:
```
print(sorted(df['Gender'].dropna().unique()))
```

```
['Female', 'Male']
```

[30]:
```
df['LoanAmount'].describe()
```

[30]:
```
count    362.000000
mean     136.132597
std       61.366652
min       28.000000
25%      100.250000
50%      125.000000
75%      158.000000
max      550.000000
Name: LoanAmount, dtype: float64
```

[31]:
```
# Assume LoanAmount > 500 is entered in a different unit
df.loc[df['LoanAmount'] > 500, 'LoanAmount'] = (
    df.loc[df['LoanAmount'] > 500, 'LoanAmount'] / 10
)

df['LoanAmount'].describe()
```

[31]:
```
count    362.000000
mean     134.765193
std       57.513025
min       28.000000
25%      100.000000
50%      125.000000
75%      157.750000
max      460.000000
Name: LoanAmount, dtype: float64
```

[32]:
```
print(df.columns)
```

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area'],
      dtype='object')
```

[33]:
```
df.drop(columns=['Loan_ID'], inplace=True)
df.head()
```

[33]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.0	Urban
1	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.0	Urban
2	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.0	Urban
3	Male	Yes	2	Graduate	No	2340	2546	100.0	360.0	NaN	Urban
4	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.0	Urban

[34]:
```
df.isnull().sum()
```

[34]:
```
Gender              0
Married             0
Dependents         10
Education           0
Self_Employed      23
ApplicantIncome     0
CoapplicantIncome   0
LoanAmount          5
Loan_Amount_Term    6
Credit_History     29
Property_Area       0
dtype: int64
```

[35]:
```
# Fill Self_Employed with mode
mode_self_employed = df['Self_Employed'].mode()[0]
df['Self_Employed'] = df['Self_Employed'].fillna(mode_self_employed)

# Fill Credit_History with mode
mode_credit = df['Credit_History'].mode()[0]
df['Credit_History'] = df['Credit_History'].fillna(mode_credit)
```

```
mode_gender = df['Gender'].mode()[0]
df['Gender'] = df['Gender'].fillna(mode_gender)

mode_dependents = df['Dependents'].mode()[0]
df['Dependents'] = df['Dependents'].fillna(mode_dependents)

mode_amount = df['LoanAmount'].mode()[0]
df['LoanAmount'] = df['LoanAmount'].fillna(mode_amount)

mode_term= df['Loan_Amount_Term'].mode()[0]
df['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(mode_term)
# Check missing values
print(df.isnull().sum())
```

```
Gender              0
Married             0
Dependents          0
Education           0
Self_Employed       0
ApplicantIncome     0
CoapplicantIncome   0
LoanAmount          0
Loan_Amount_Term    0
Credit_History      0
Property_Area       0
dtype: int64
```

[36]:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

df[['ApplicantIncome_MinMax', 'LoanAmount_MinMax']] = scaler.fit_transform(
    df[['ApplicantIncome', 'LoanAmount']]
)
```

```
df[['ApplicantIncome_MinMax', 'LoanAmount_MinMax']].head()
```

[36]:

	ApplicantIncome_MinMax	LoanAmount_MinMax
0	0.078865	0.189815
1	0.042411	0.226852
2	0.068938	0.416667
3	0.032263	0.166667
4	0.045168	0.115741

[4]:

```
#Method 1 : Mean Imputation

df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].mean())
print(df[['LoanAmount']].head(20))
```

```
    LoanAmount
0        110.0
1        126.0
2        208.0
3        100.0
4         78.0
5        152.0
6         59.0
7        147.0
8        280.0
9        123.0
10        90.0
11       162.0
12        40.0
13       166.0
```

jupyter Loan prediction Last Checkpoint: 9 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab

[5]:
```
# Method 2: Median Imputation
# Replace missing values with the middle value

df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].median())

print(df[['LoanAmount']].head(20))
```

```
    LoanAmount
0        110.0
1        126.0
2        208.0
3        100.0
4         78.0
5        152.0
6         59.0
7        147.0
8        280.0
9        123.0
10        90.0
11       162.0
12        40.0
13       166.0
14       124.0
15       131.0
16       200.0
17       126.0
18       300.0
19       100.0
```

[6]:
```
# Method 3: Mode Imputation
# Replace missing values with most frequent value

df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])

print(df[['Gender']].head(20))
```

```
print(df[['Gender']].head(20))
```

```
    Gender
0     Male
1     Male
2     Male
3     Male
4     Male
5     Male
6   Female
7     Male
8     Male
9     Male
10    Male
11    Male
12    Male
13    Male
14  Female
15    Male
16    Male
17    Male
18    Male
19    Male
```

[8]:

```
# Check missing values before filling
print(df.isnull().sum())

# Forward Fill (FFill)
df = df.ffill()

# Check missing values after filling
print(df.isnull().sum())

# Display first 20 rows
print(df.head(20))
```

```
Loan_ID            0
```

```
Loan_ID              0
Gender               0
Married              0
Dependents           0
Education            0
Self_Employed        0
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount           0
Loan_Amount_Term     0
Credit_History       0
Property_Area        0
dtype: int64
Loan_ID              0
Gender               0
Married              0
Dependents           0
Education            0
Self_Employed        0
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount           0
Loan_Amount_Term     0
Credit_History       0
Property_Area        0
dtype: int64
     Loan_ID  Gender Married Dependents     Education Self_Employed  \
0   LP001015    Male     Yes          0      Graduate            No
1   LP001022    Male     Yes          1      Graduate            No
2   LP001031    Male     Yes          2      Graduate            No
3   LP001035    Male     Yes          2      Graduate            No
4   LP001051    Male      No          0  Not Graduate            No
5   LP001054    Male     Yes          0  Not Graduate           Yes
6   LP001055  Female      No          1  Not Graduate            No
7   LP001056    Male     Yes          2  Not Graduate            No
8   LP001059    Male     Yes          2      Graduate            No
```

2	5000	1800	208.0	360.0
3	2340	2546	100.0	360.0
4	3276	0	78.0	360.0
5	2165	3422	152.0	360.0
6	2226	0	59.0	360.0
7	3881	0	147.0	360.0
8	13633	0	280.0	240.0
9	2400	2400	123.0	360.0
10	3091	0	90.0	360.0
11	2185	1516	162.0	360.0
12	4166	0	40.0	180.0
13	12173	0	166.0	360.0
14	4666	0	124.0	360.0
15	5667	0	131.0	360.0
16	4583	2916	200.0	360.0
17	3786	333	126.0	360.0
18	9226	7916	300.0	360.0
19	1300	3470	100.0	180.0

	Credit_History	Property_Area
0	1.0	Urban
1	1.0	Urban
2	1.0	Urban
3	1.0	Urban
4	1.0	Urban
5	1.0	Urban
6	1.0	Semiurban
7	0.0	Rural
8	1.0	Urban
9	1.0	Semiurban
10	1.0	Urban
11	1.0	Semiurban
12	1.0	Urban
13	0.0	Semiurban
14	1.0	Semiurban
15	1.0	Urban

jupyter Loan prediction Last Checkpoint: 10 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python [conc

[9]:
```
from sklearn.impute import KNNImputer

# Select numerical columns
num_cols = ['LoanAmount', 'Loan_Amount_Term', 'Credit_History']

# Apply KNN Imputation
imputer = KNNImputer(n_neighbors=3)
df[num_cols] = imputer.fit_transform(df[num_cols])

# Check missing values
print(df[num_cols].isnull().sum())
```

```
LoanAmount          0
Loan_Amount_Term    0
Credit_History      0
dtype: int64
```

[10]:
```
from sklearn.impute import KNNImputer
import pandas as pd

# Select numerical columns
num_cols = ['LoanAmount', 'Loan_Amount_Term', 'Credit_History']

# Apply KNN Imputation
knn = KNNImputer(n_neighbors=3)

df[num_cols] = knn.fit_transform(df[num_cols])

print(df[num_cols].head())
```

```
   LoanAmount  Loan_Amount_Term  Credit_History
0       110.0             360.0             1.0
1       126.0             360.0             1.0
2       208.0             360.0             1.0
3       100.0             360.0             1.0
```

jupyter Loan prediction Last Checkpoint: 11 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab

[13]:
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("test_Y3wMUE5_7gLdaTN.csv")

le = LabelEncoder()

df['Gender'] = le.fit_transform(df['Gender'].astype(str))
df['Married'] = le.fit_transform(df['Married'].astype(str))
df['Education'] = le.fit_transform(df['Education'].astype(str))
df['Self_Employed'] = le.fit_transform(df['Self_Employed'].astype(str))

df = pd.get_dummies(df, columns=['Property_Area'])

print(df.head())
```

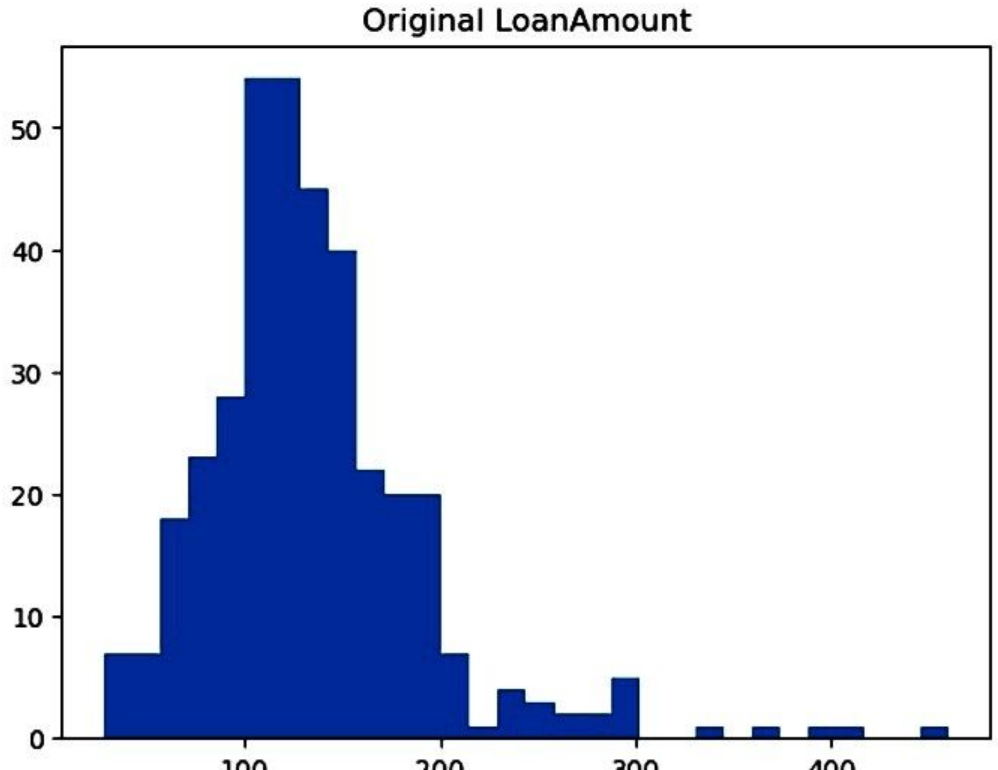
```
    Loan_ID  Gender  Married Dependents  Education  Self_Employed  \
0  LP001015       1        1          0          0              0
1  LP001022       1        1          1          0              0
2  LP001031       1        1          2          0              0
3  LP001035       1        1          2          0              0
4  LP001051       1        0          0          1              0

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             5720                  0       110.0             360.0
1             3076               1500       126.0             360.0
2             5000               1800       208.0             360.0
3             2340               2546       100.0             360.0
4             3276                  0        78.0             360.0

   Credit_History  Property_Area_Rural  Property_Area_Semiurban  \
0             1.0                False                    False
1             1.0                False                    False
2             1.0                False                    False
3             NaN                False                    False
4             1.0                False                    False
```

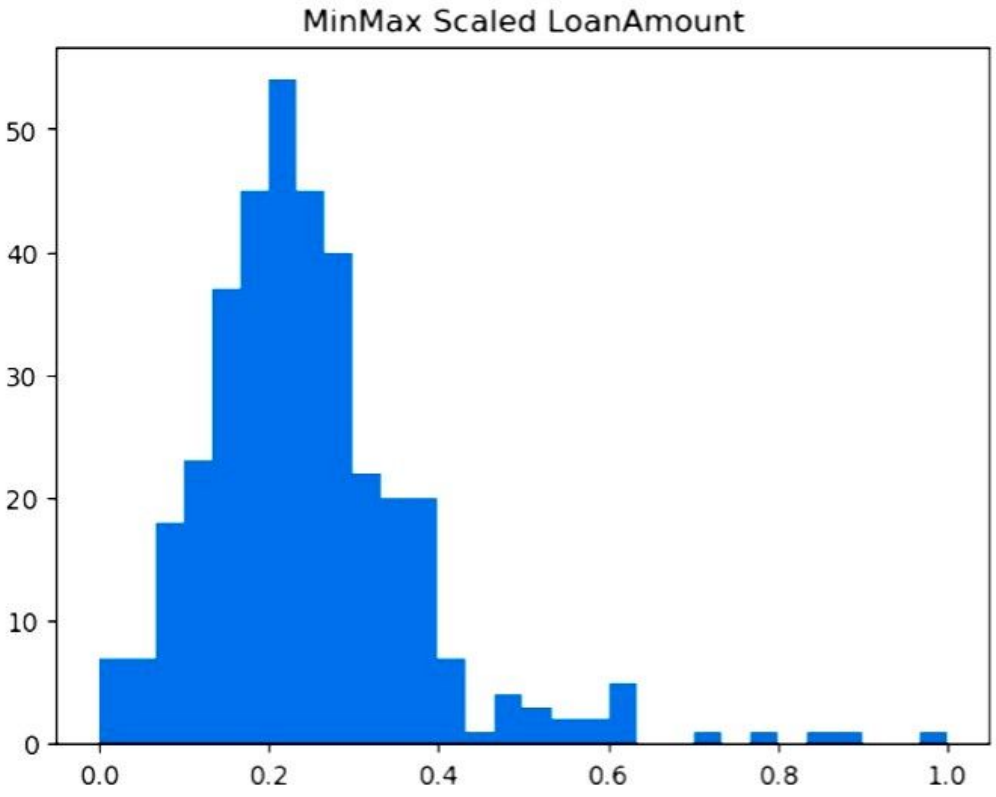
jupyter Loan prediction Last Checkpoint: 11 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python [co

[37]:
```
import matplotlib.pyplot as plt

# Original LoanAmount
plt.hist(df['LoanAmount'].dropna(), bins=30)
plt.title("Original LoanAmount")
plt.show()

# Scaled LoanAmount
plt.hist(df['LoanAmount_MinMax'], bins=30)
plt.title("MinMax Scaled LoanAmount")
plt.show()
```






```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df[['ApplicantIncome_Std', 'LoanAmount_Std']] = scaler.fit_transform(
    df[['ApplicantIncome', 'LoanAmount']]
)

df[['ApplicantIncome_Std', 'LoanAmount_Std']].head()
```

```
df[['ApplicantIncome_Std', 'LoanAmount_Std']].head()
```

[38]:

	ApplicantIncome_Std	LoanAmount_Std
0	0.186461	-0.437594
1	-0.352692	-0.157228
2	0.039641	1.279647
3	-0.502774	-0.612823
4	-0.311909	-0.998326

[39]:

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()

df[['ApplicantIncome_Robust', 'LoanAmount_Robust']] = scaler.fit_transform(
    df[['ApplicantIncome', 'LoanAmount']]
)

df[['ApplicantIncome_Robust', 'LoanAmount_Robust']].head()
```

[39]:

	ApplicantIncome_Robust	LoanAmount_Robust
0	0.880692	-0.270270
1	-0.323315	0.018018
2	0.552823	1.495495
3	-0.658470	-0.450450
4	-0.232240	-0.846847

[ ]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import zscore

# Box Plot
sns.boxplot(x=df['LoanAmount'])
plt.show()

# IQR Method
Q1 = df['LoanAmount'].quantile(0.25)
Q3 = df['LoanAmount'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers_iqr = df[
    (df['LoanAmount'] < lower) |
    (df['LoanAmount'] > upper)
]

print("IQR Outliers:", len(outliers_iqr))

# Z-Score Method
df['LoanAmount_zscore'] = zscore(df['LoanAmount'])

outliers_z = df[
    abs(df['LoanAmount_zscore']) > 3
]

print("Z-Score Outliers:", len(outliers_z))
```

jupyter Loan prediction Last Checkpoint: 12 minutes ago

ile Edit View Run Kernel Settings Help

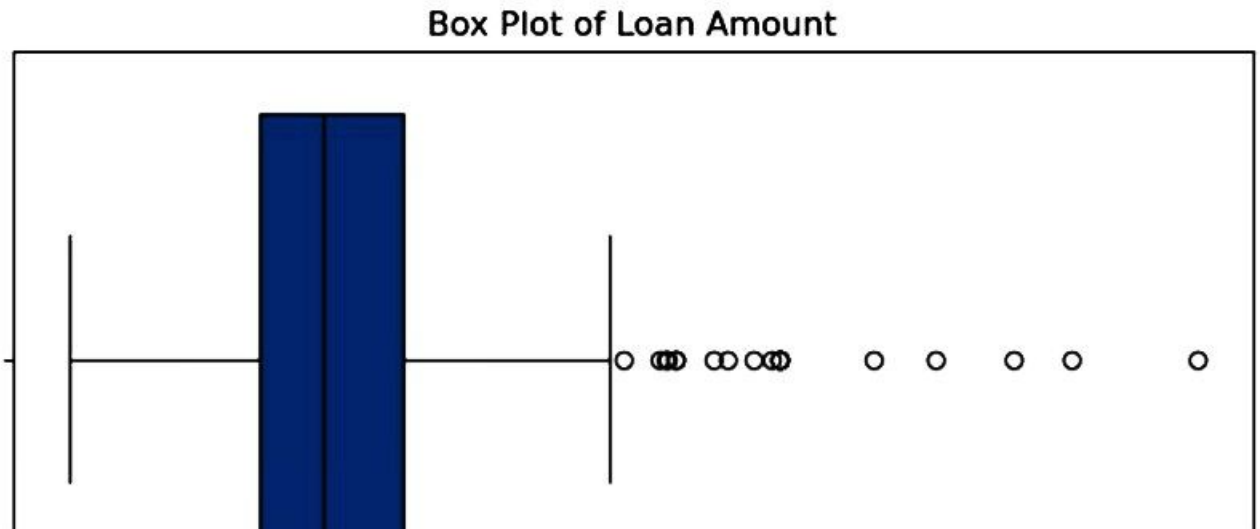
Code JupyterLab

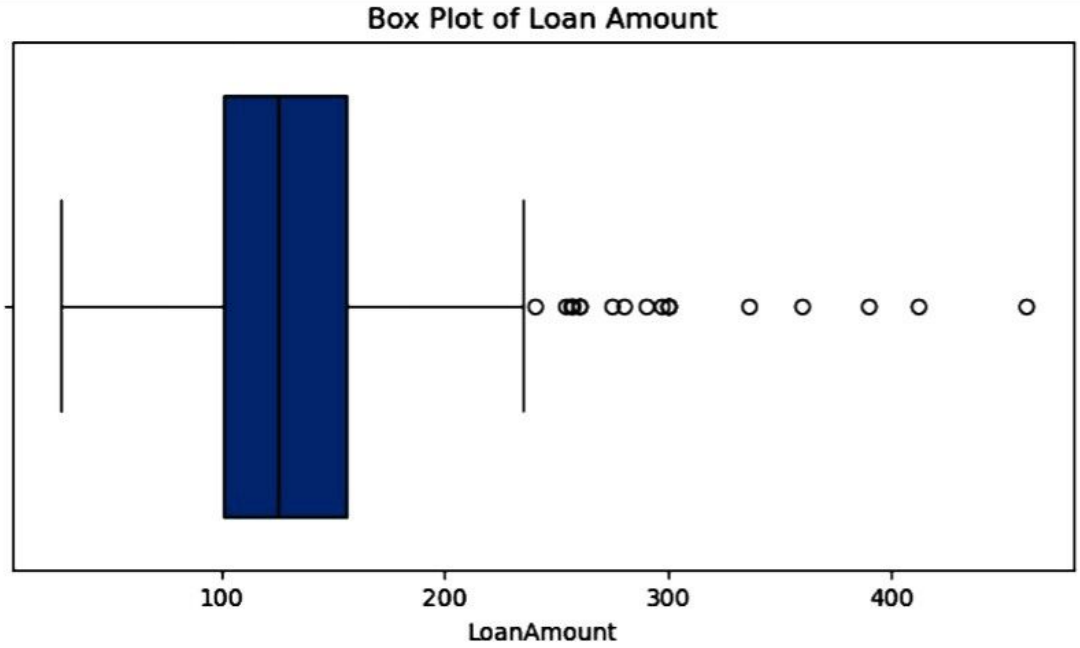
EDA

[43]:
```
# Descriptive Statistics
print(df['LoanAmount'].describe())
```

```
count    367.000000
mean     134.972752
std       57.146197
min       28.000000
25%      100.500000
50%      125.000000
75%      156.000000
max      460.000000
Name: LoanAmount, dtype: float64
```

[44]:
```
### Box plot
plt.figure(figsize=(8,4))
sns.boxplot(x=df['LoanAmount'])
plt.title("Box Plot of Loan Amount")
plt.show()
```



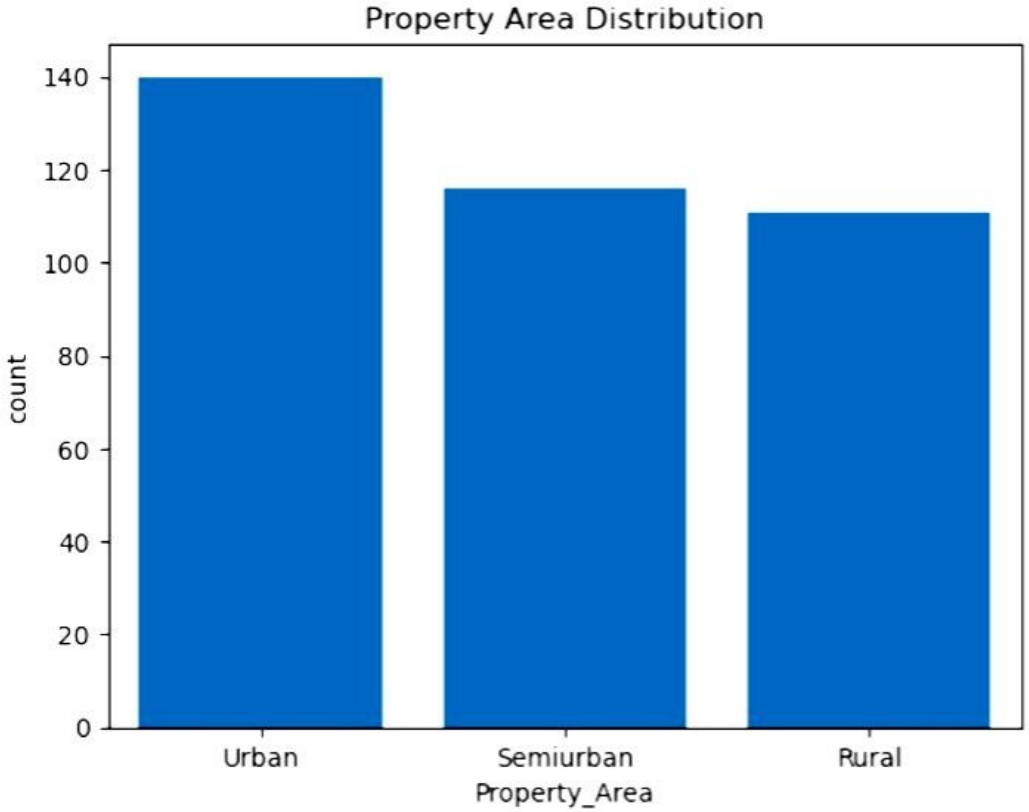


[46]:
```
##  Categorical Variable: Day
print(df['Gender'].value_counts())
```

```
Gender
Male      297
Female     70
Name: count, dtype: int64
```

[48]:
```
sns.countplot(x='Property_Area', data=df)
plt.title("Property Area Distribution")
plt.show()
```

```
plt.show()
```




```
df['Gender'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%'
)

plt.title("Gender Distribution")
plt.ylabel("")
plt.show()
```

jupyter Loan prediction Last Checkpoint: 13 minutes ago

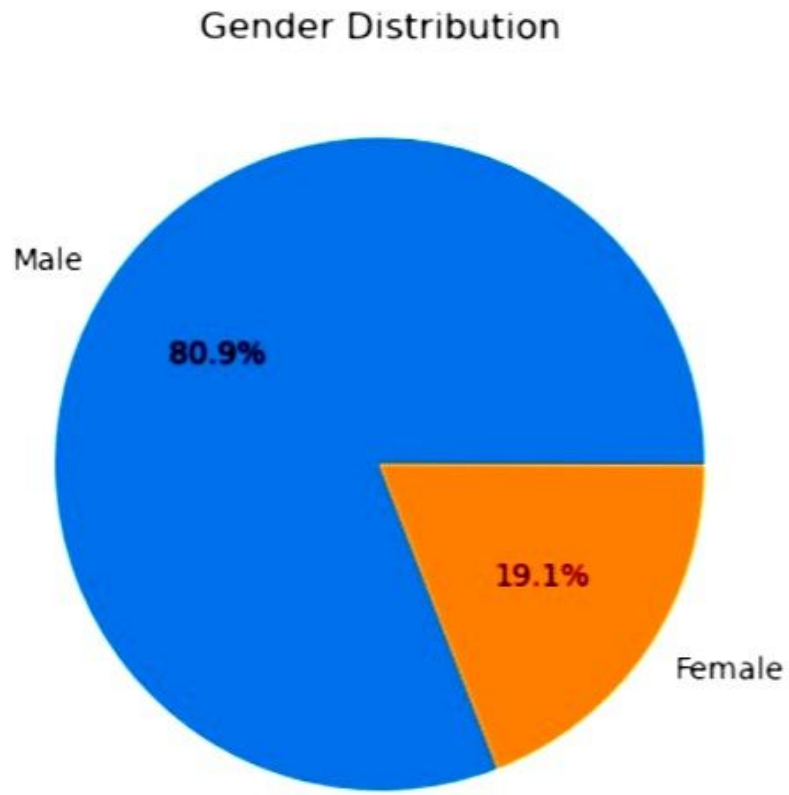
File Edit View Run Kernel Settings Help

Code

Jupyter

Property_Area

```
df['Gender'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%'
)

plt.title("Gender Distribution")
plt.ylabel("")
plt.show()
```



jupyter Loan prediction Last Checkpoint: 13 minutes ago

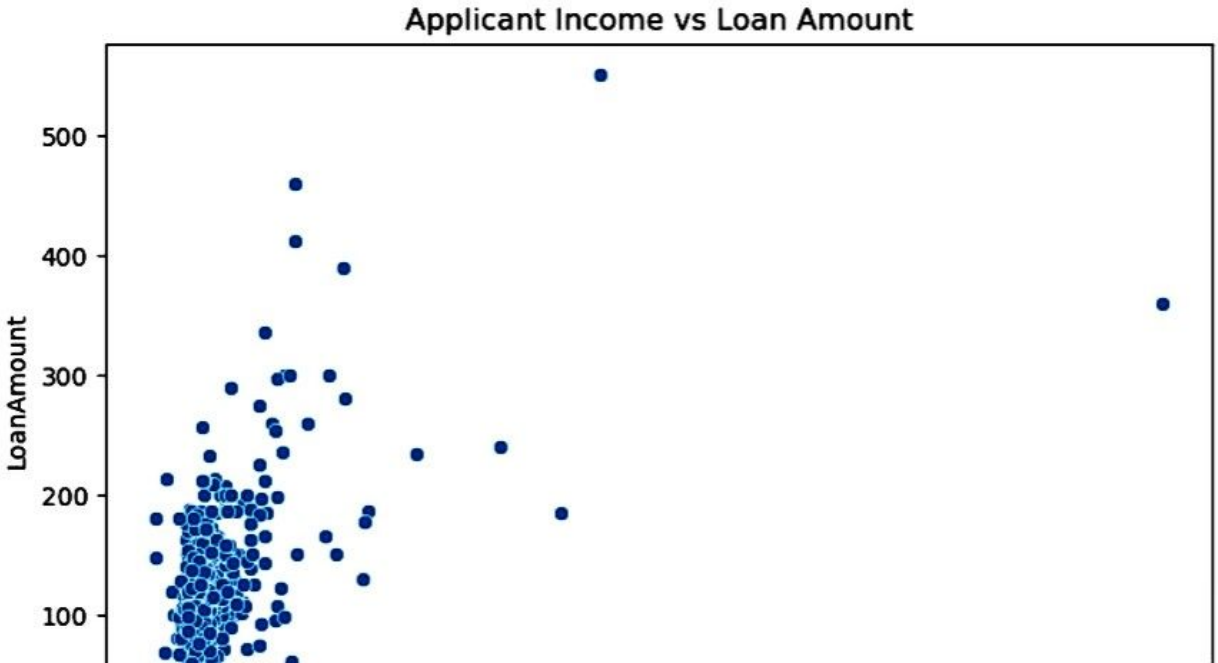
File Edit View Run Kernel Settings Help

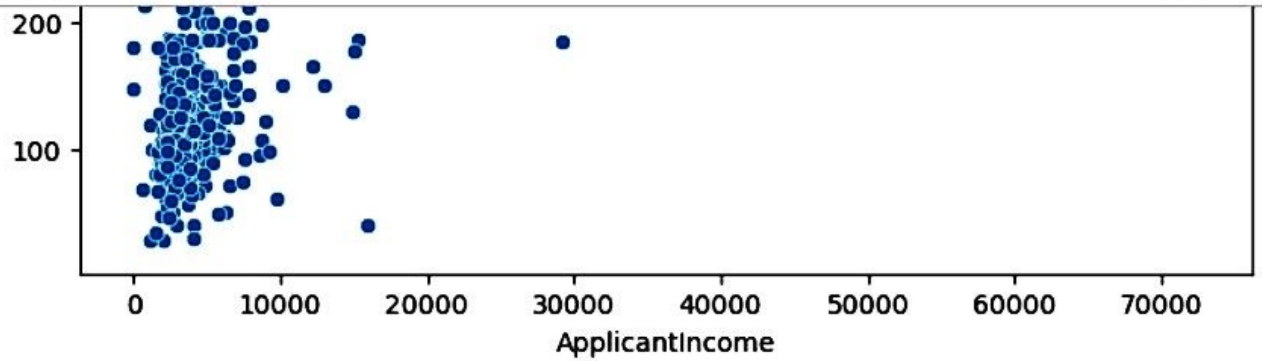
Code JupyterLab Pyth

[14]:
```
##Outlier Detection using IQR
# Outlier Detection using IQR

Q1 = df['LoanAmount'].quantile(0.25)
Q3 = df['LoanAmount'].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[
    (df['LoanAmount'] < lower) |
    (df['LoanAmount'] > upper)
]

print("Outliers:", len(outliers))
print(outliers[['LoanAmount']])
```

```
Outliers: 18
     LoanAmount
8         280.0
18        300.0
24        290.0
27        275.0
81        360.0
83        257.0
91        390.0
96        256.0
124       300.0
143       550.0
144       260.0
189       336.0
194       412.0
```

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

sns.scatterplot(
    x='ApplicantIncome',
    y='LoanAmount',
    data=df
)

plt.title("Applicant Income vs Loan Amount")
plt.show()
```



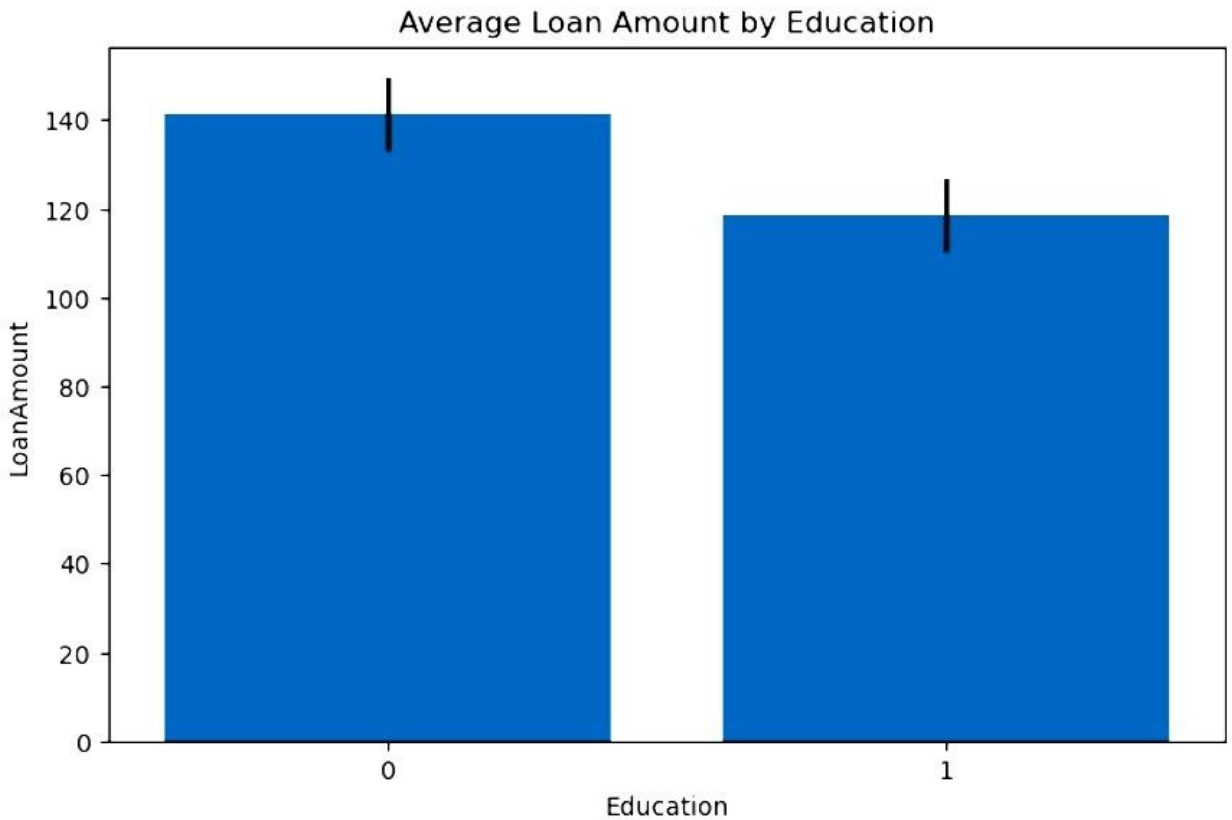


[17]:
```
# Pearson Correlation

corr = df['ApplicantIncome'].corr(df['LoanAmount'])

print("Correlation:", corr)
```

```
Correlation: 0.4934505782156068
```

[19]:
```
## Categorical vs Numerical
# bar chart
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

sns.barplot(
    x='Education',
    y='LoanAmount',
    data=df
)

plt.title("Average Loan Amount by Education")
plt.show()
```

```
plt.title("Average Loan Amount by Education")
plt.show()
```

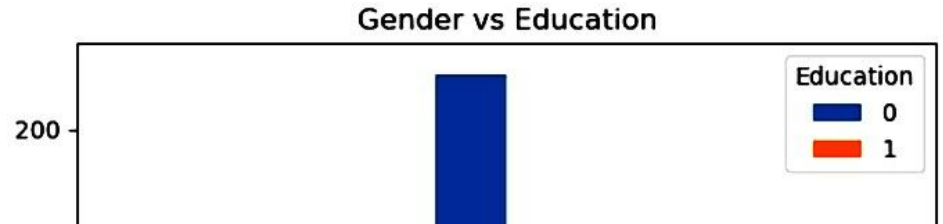



```
# Mean Loan Amount by education
print(
    df.groupby('Education', observed=False)['LoanAmount'].mean()
)
```

[20]:
```
# Mean Loan Amount by education
print(
    df.groupby('Education', observed=False)['LoanAmount'].mean()
)
```

```
Education
0    141.358423
1    118.566265
Name: LoanAmount, dtype: float64
```

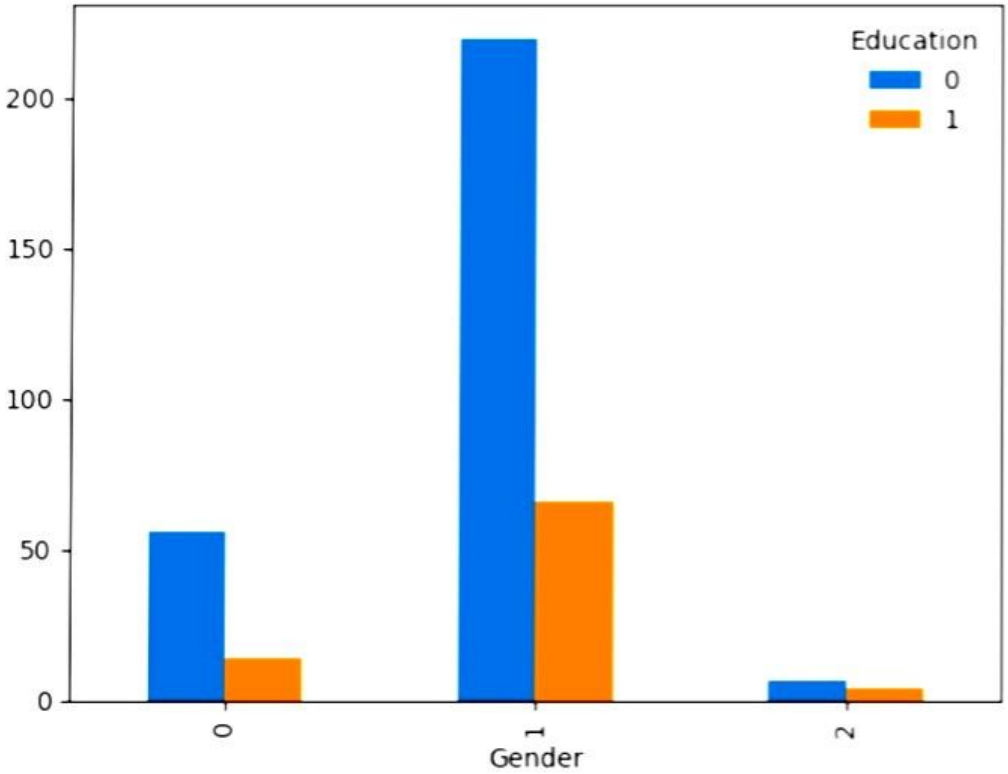
[22]:
```
## Cross tab
import pandas as pd
import matplotlib.pyplot as plt

pd.crosstab(
    df['Gender'],
    df['Education']
).plot(kind='bar')

plt.title("Gender vs Education")
plt.show()
```



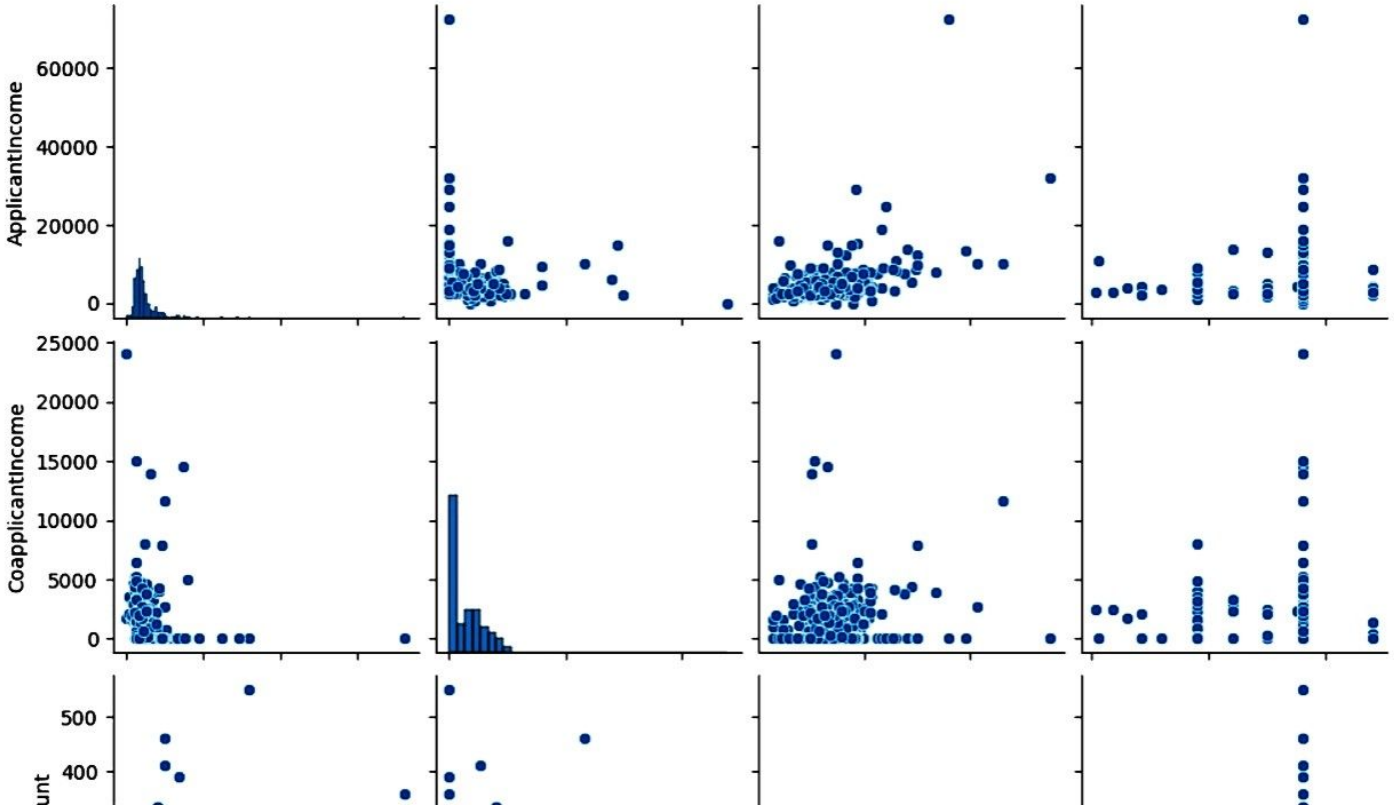
jupyter Loan prediction Last Checkpoint: 18 minutes ago

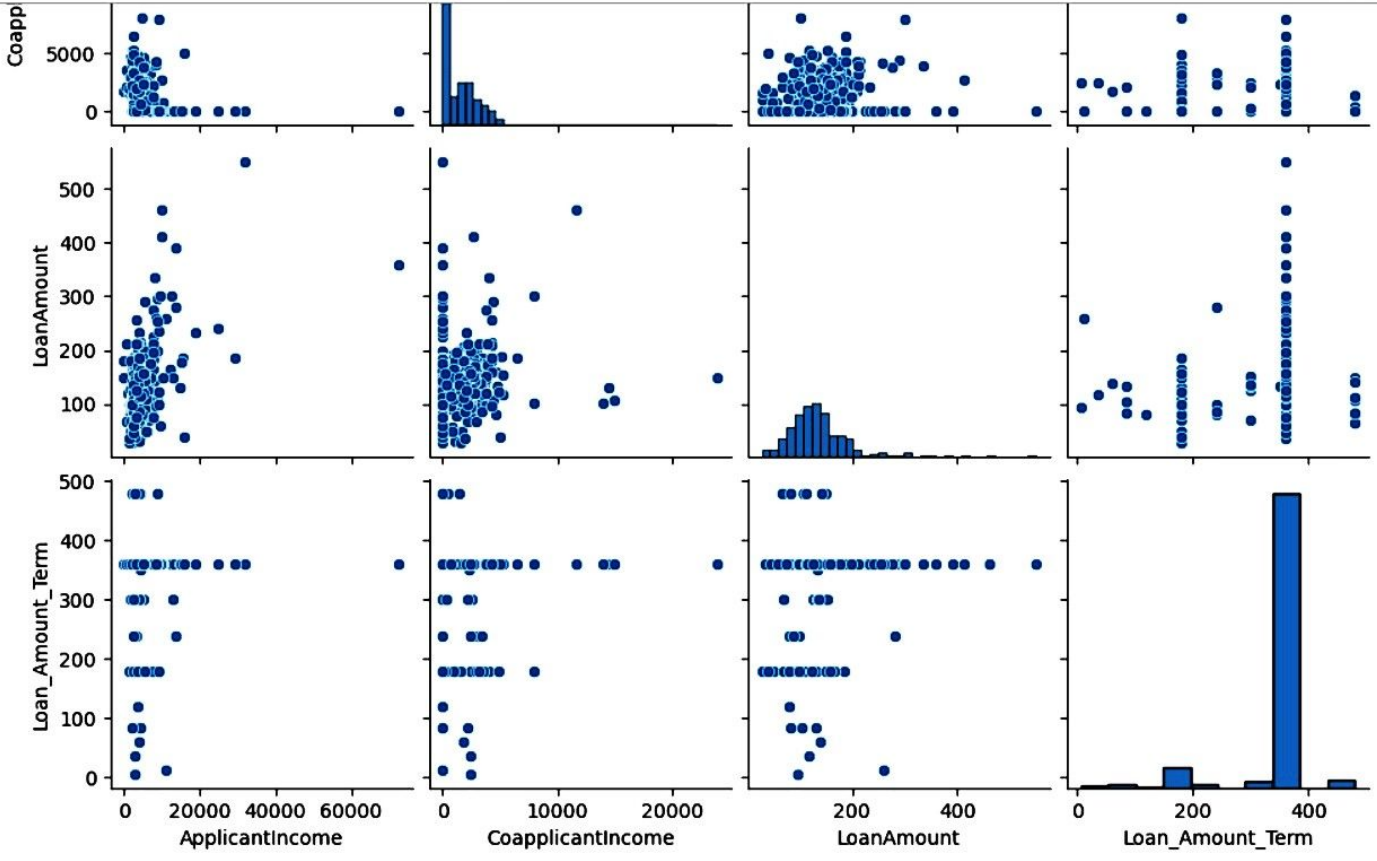
ile Edit View Run Kernel Settings Help

+ Code JupyterLab Python [co

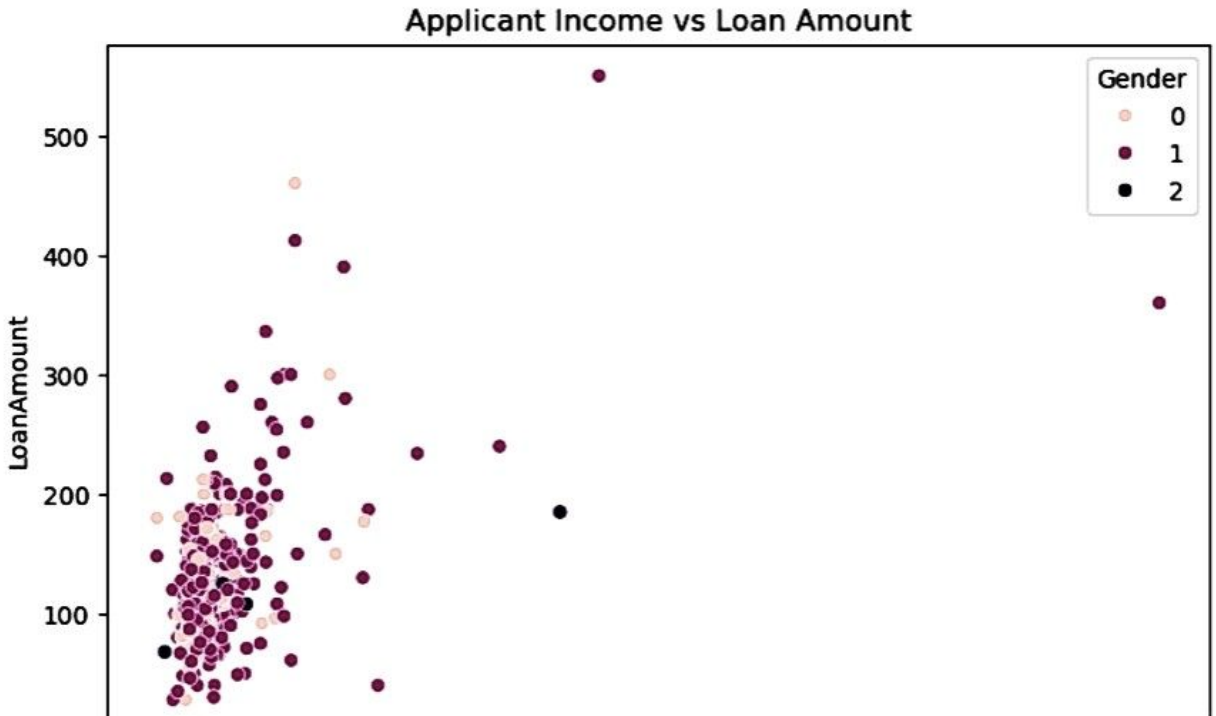


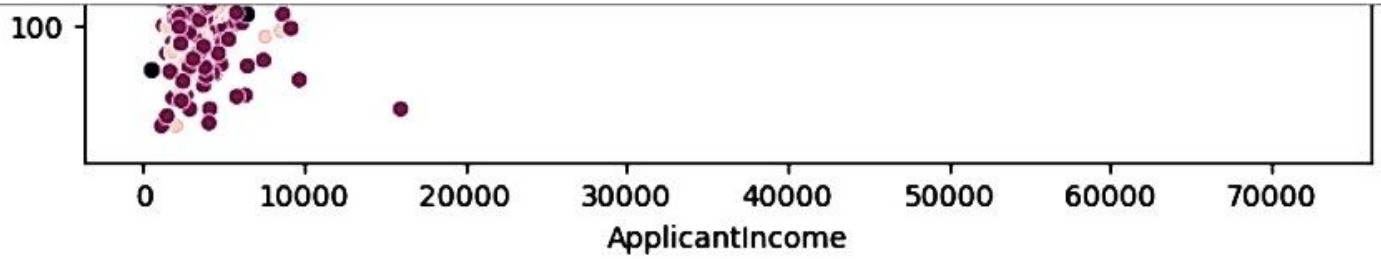
```
)

plt.show()
```


ApplicantIncome
60000
40000
20000
0
CoapplicantIncome
25000
20000
15000
10000
5000
0
500
400


Coapp
5000
0
LoanAmount
500
400
300
200
100
Loan_Amount_Term
500
400
300
200
100
0
0
20000
40000
60000
ApplicantIncome
0
10000
20000
CoapplicantIncome
200
400
LoanAmount
0
200
400
Loan_Amount_Term

```
plt.figure(figsize=(8,5))

sns.scatterplot(
    x='ApplicantIncome',
    y='LoanAmount',
    hue='Gender',
    data=df
)

plt.title("Applicant Income vs Loan Amount")
plt.show()
```





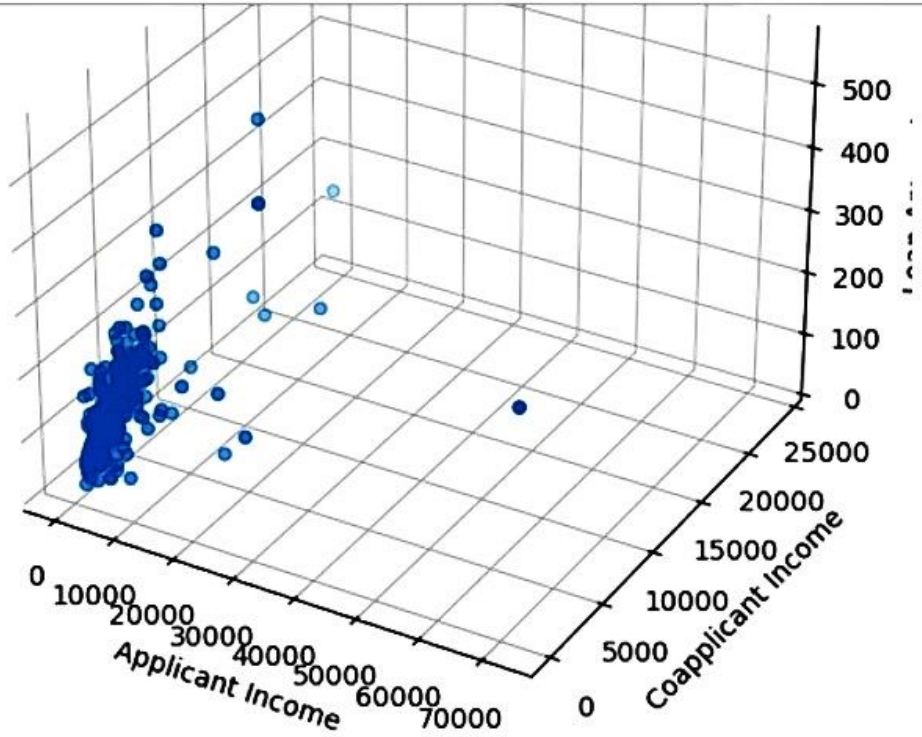
[25]:

```
# scatter plot
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(
    df['ApplicantIncome'],
    df['CoapplicantIncome'],
    df['LoanAmount']
)

ax.set_xlabel('Applicant Income')
ax.set_ylabel('Coapplicant Income')
ax.set_zlabel('Loan Amount')

plt.title("3D Scatter Plot of Loan Prediction Data")
plt.show()
```

jupyter Loan prediction Last Checkpoint: 20 minutes ago

le Edit View Run Kernel Settings Help

+ ✂ ⧉ 📋 ▶ ■ ↻ ▶▶ Code JupyterLab Pyth



[26]:

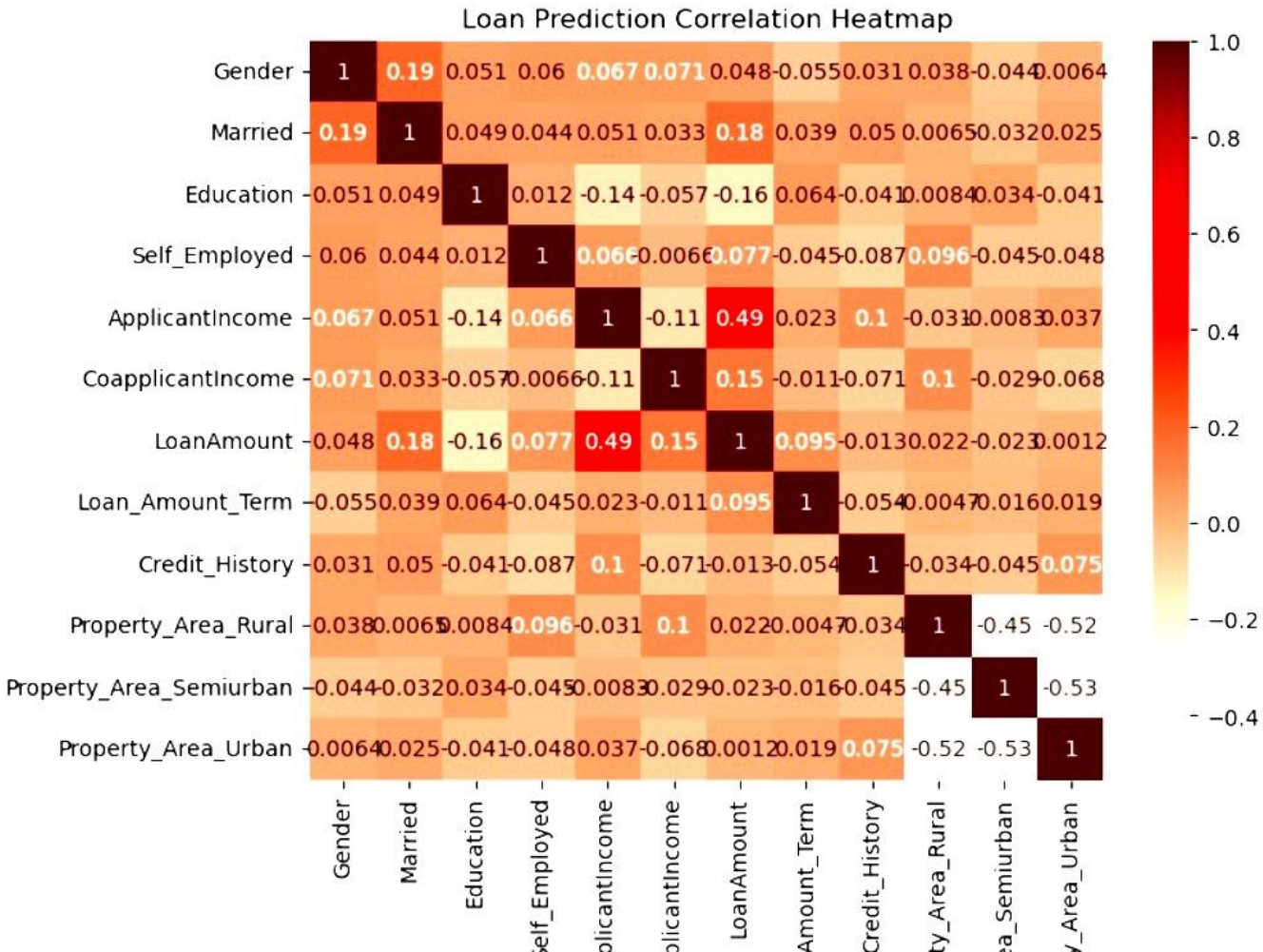
```
### Heat map
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,6))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap='Reds'
)

plt.title("Loan Prediction Correlation Heatmap")
```

jupyter Loan prediction Last Checkpoint: 20 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python [conda env:base] *

```
plt.show()
```



[29]:
```
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = df[
    [
        'ApplicantIncome',
        'CoapplicantIncome',
        'LoanAmount',
        'Loan_Amount_Term',
        'Credit_History'
    ]
]

# Remove rows with missing values
X = X.dropna()

vif = pd.DataFrame()
vif["Feature"] = X.columns

vif["VIF"] = [
    variance_inflation_factor(X.values, i)
    for i in range(X.shape[1])
]

print(vif)
```

```
            Feature       VIF
0    ApplicantIncome  2.538849
1  CoapplicantIncome  1.554684
2         LoanAmount  8.036626
3   Loan_Amount_Term  8.597486
4     Credit_History  4.890163
```

jupyter Loan prediction Last Checkpoint: 21 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python [conda env:base] *

[30]:
```
from statsmodels.stats.outliers_influence import variance_inflation_factor
import pandas as pd

X_new = X.drop('ApplicantIncome', axis=1)

vif_after = pd.DataFrame()

vif_after["Feature"] = X_new.columns

vif_after["VIF"] = [
    variance_inflation_factor(X_new.values, i)
    for i in range(X_new.shape[1])
]

print(vif_after)
```

```
             Feature       VIF
0  CoapplicantIncome  1.491516
1         LoanAmount  6.191115
2   Loan_Amount_Term  8.562274
3     Credit_History  4.846311
```

[31]:
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,6))

sns.heatmap(
    X_new.corr(),
    annot=True,
    cmap='Greens'
)

plt.title("Correlation Heatmap After VIF Treatment")
plt.show()
```

jupyter Loan prediction Last Checkpoint: 21 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python [conda env:base]